```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
df = pd.read_csv("/content/water_potability.csv")
df.head()
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3276 entries, 0 to 3275
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   ph               2785 non-null   float64
 1   Hardness         3276 non-null   float64
 2   Solids           3276 non-null   float64
 3   Chloramines      3276 non-null   float64
 4   Sulfate          2495 non-null   float64
 5   Conductivity     3276 non-null   float64
 6   Organic_carbon   3276 non-null   float64
 7   Trihalomethanes  3114 non-null   float64
 8   Turbidity        3276 non-null   float64
 9   Potability       3276 non-null   int64
dtypes: float64(9), int64(1)
memory usage: 256.1 KB
```

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trihalomethanes	Turbidity
count	2785.000000	3276.000000	3276.000000	3276.000000	2495.000000	3276.000000	3276.000000	3114.000000	3276.00000
mean	7.080795	196.369496	22014.092526	7.122277	333.775777	426.205111	14.284970	66.396293	3.966786
std	1.594320	32.879761	8768.570828	1.583085	41.416840	80.824064	3.308162	16.175008	0.780382
min	0.000000	47.432000	320.942611	0.352000	129.000000	181.483754	2.200000	0.738000	1.450000
25%	6.093092	176.850538	15666.690297	6.127421	307.699498	365.734414	12.065801	55.844536	3.439711
50%	7.036752	196.967627	20927.833607	7.130299	333.073546	421.884968	14.218338	66.622485	3.955028
75%	8.062066	216.667456	27332.762127	8.114887	359.950170	481.792304	16.557652	77.337473	4.500320
max	14.000000	323.124000	61227.196008	13.127000	481.030642	753.342620	28.300000	124.000000	6.739000

```
df = df.fillna(df.mean())
```

```
X = df.drop("Potability", axis=1)
y = df["Potability"]
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
▾ LogisticRegression
LogisticRegression(max_iter=1000)
```

```
y_pred = model.predict(X_test)
```

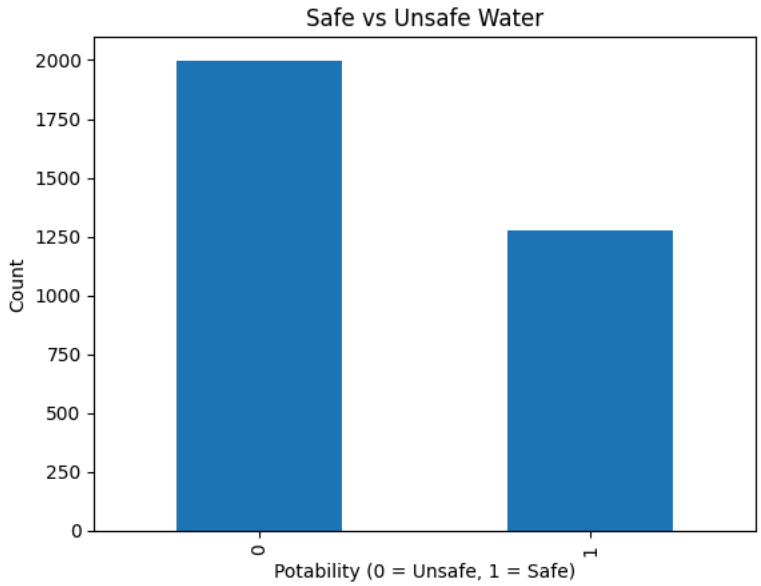
```
from sklearn.metrics import accuracy_score

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 0.6280487804878049
```

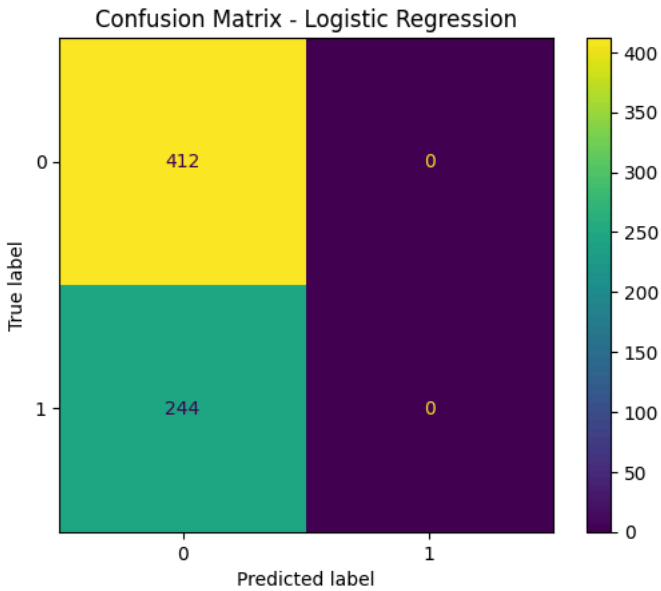
McAfee | WebAdvisor
Your download's being scanned.
We'll let you know if there's an issue.

```
import matplotlib.pyplot as plt
```

```
df['Potability'].value_counts().plot(kind='bar')

plt.title("Safe vs Unsafe Water")
plt.xlabel("Potability (0 = Unsafe, 1 = Safe)")
plt.ylabel("Count")

plt.show()
```

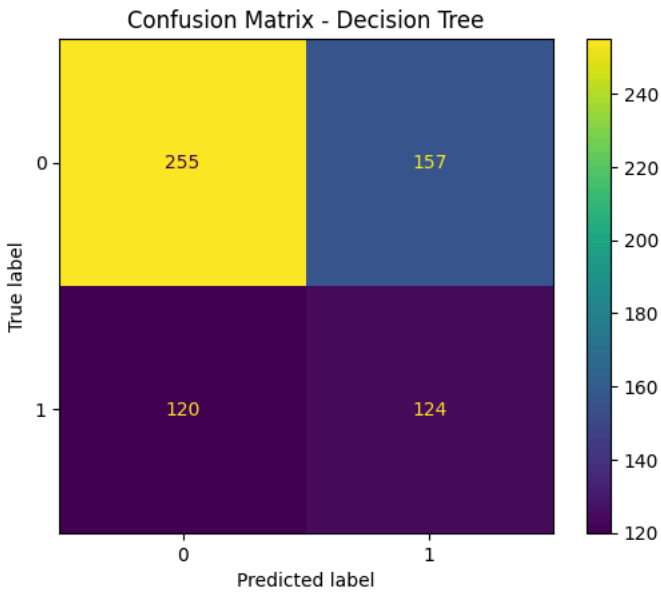



```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()

plt.title("Confusion Matrix - Logistic Regression")
plt.show()
```

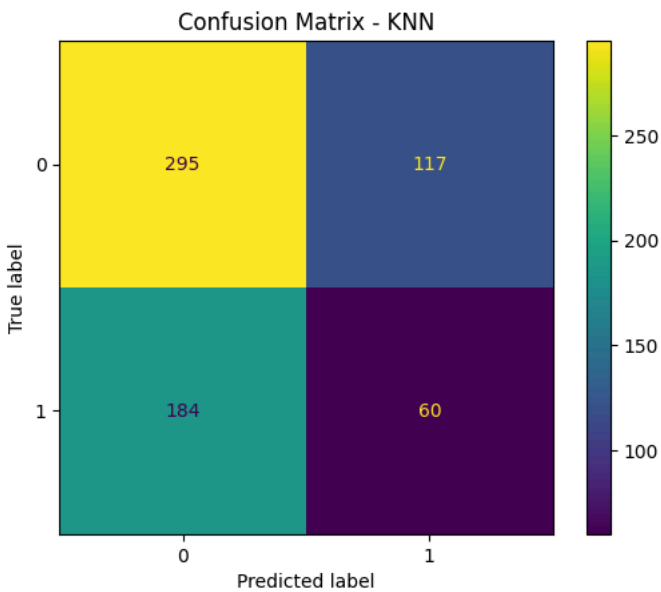



```
cm_dt = confusion_matrix(y_test, dt_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm_dt)
disp.plot()

plt.title("Confusion Matrix - Decision Tree")
plt.show()
```

McAfee | WebAdvisor
Your download's being scanned.
We'll let you know if there's an issue.



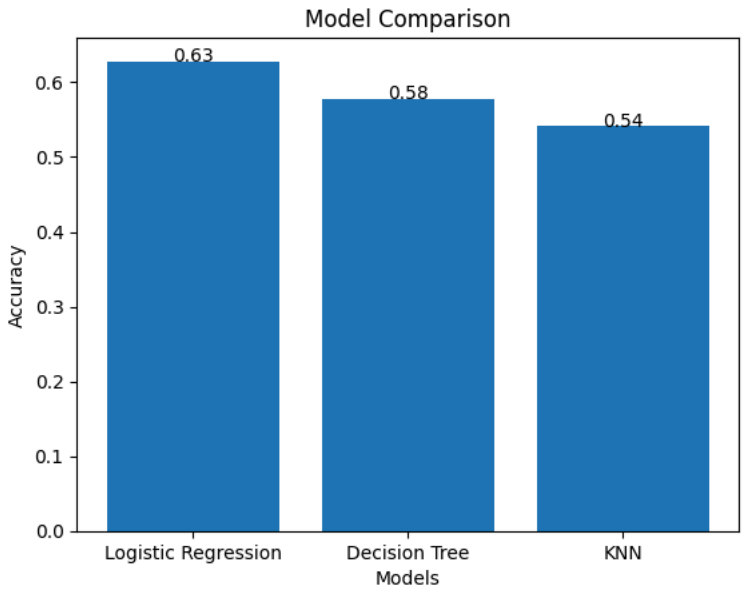

```
cm_knn = confusion_matrix(y_test, knn_pred)

disp = ConfusionMatrixDisplay(confusion_matrix=cm_knn)
disp.plot()

plt.title("Confusion Matrix - KNN")
plt.show()
```




```
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

# store accuracies
lr_acc = accuracy_score(y_test, y_pred)
dt_acc = accuracy_score(y_test, dt_pred)
knn_acc = accuracy_score(y_test, knn_pred) # Now knn_pred should be defined

models = ['Logistic Regression', 'Decision Tree', 'KNN']
accuracies = [lr_acc, dt_acc, knn_acc]

plt.figure()
plt.bar(models, accuracies)

# show values on top
for i, v in enumerate(accuracies):
    plt.text(i, v, str(round(v, 2)), ha='center')
```

McAfee | WebAdvisor
Your download's being scanned.
We'll let you know if there's an issue.

```
plt.title("Model Comparison")
plt.xlabel("Models")
plt.ylabel("Accuracy")

plt.show()
```




```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

print("---- Logistic Regression ----")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("Recall:", recall_score(y_test, y_pred))
print("F1 Score:", f1_score(y_test, y_pred))

print("\n---- Decision Tree ----")
print("Accuracy:", accuracy_score(y_test, dt_pred))
print("Precision:", precision_score(y_test, dt_pred))
print("Recall:", recall_score(y_test, dt_pred))
print("F1 Score:", f1_score(y_test, dt_pred))

print("\n---- KNN ----")
print("Accuracy:", accuracy_score(y_test, knn_pred))
print("Precision:", precision_score(y_test, knn_pred))
print("Recall:", recall_score(y_test, knn_pred))
print("F1 Score:", f1_score(y_test, knn_pred))
```

```
---- Logistic Regression ----
Accuracy: 0.6280487804878049
Precision: 0.0
Recall: 0.0
F1 Score: 0.0

---- Decision Tree ----
Accuracy: 0.5777439024390244
Precision: 0.4412811387900356
Recall: 0.5081967213114754
F1 Score: 0.4723809523809524

---- KNN ----
Accuracy: 0.5411585365853658
Precision: 0.3389830508474576
Recall: 0.2459016393442623
F1 Score: 0.2850356294536817
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-define
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

```
sample = [[7.0, 180, 20000, 7.0, 300, 400, 10, 80, 4]]

prediction = model.predict(sample)

if prediction[0] == 1:
```

McAfee | WebAdvisor
Your download's being scanned.
We'll let you know if there's an issue.

```
    print("Safe Water")
else:
    print("Unsafe Water")
```

```
Unsafe Water
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but
  warnings.warn(
```

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)

dt_pred = dt.predict(X_test)

print("Decision Tree Accuracy:", accuracy_score(y_test, dt_pred))
```

```
Decision Tree Accuracy: 0.5777439024390244
```

McAfee | WebAdvisor
Your download's being scanned.
We'll let you know if there's an issue.